

Methodology for Large-Scale Entity Resolution Without Pairwise Matching

Cheng Chen

Black Oak Analytics, Inc.
Little Rock, AR
cchen@blackoakgroup.com

Daniel Pullen

Information Science
UALR
Little Rock, AR
dpullen@ualr.edu

Reed H. Petty

Information Science
UALR
Little Rock, AR
rhpetty@ualr.edu

John R. Talburt

Black Oak Analytics, Inc.
Little Rock, AR
jrtalburt@ualr.edu

Abstract— Entity Resolution is the process of determining if two information system records are referring to the same entities, and is a crucial part in Information Quality research. The ER process becomes exponentially more complex and time consuming as datasets approach Big Data volumes. Due to the special characters of transitive closure in Entity Resolution and high volume of input data, traditional ER pairwise matching algorithms are not able to solve the problem efficiently. This paper presents a methodology to perform Entity Resolution without pairwise matching using match keys. Transitive closure occurs when each input reference can potentially create more than one match key. This paper also introduces a novel distributed parallel transitive closure algorithm in Entity Resolution context and an optimized version, which applies the method on multiple match keys. The implementation of the methodology is built upon the Hadoop MapReduce for distributed computation.

Keywords—Entity Resolution, Transitive Closure, Hadoop

I. INTRODUCTION

Entity Resolution (ER) is often mischaracterized as a deduplication process. Deduplication is the task of identifying duplicate (or near duplicate) records in a data repository that refer to the same real world entity or object [1]. Contrasting the definition of deduplication with that of ER, we find the differentiation is subtle but significant. ER is defined as the process of determining whether two references to real world objects are to the same object or different objects [2]. References referring to the same entity are said to be equivalent references.

Fundamentally, deduplication is about finding matching records, whereas ER is about finding equivalent references. The problem is that equivalent references may not match, and matching references may not be equivalent. For example, references to “Mary Smith, 123 Oak St” and to “Mary Jones, 345 Pine St” may be equivalent. She may have married, changed her last name, and moved to a new address. Similarly, references with the same name and same address may match exactly, but not be equivalent because they are a father and son living at the same address, but the reference information failed to provide any information to discriminate between them such as generational suffix (Jr, Sr) or age.

This statement is not intended to minimize the importance of matching. Matching is a key component of ER. In their pioneering research, Ivan P. Fellegi and Alan B. Sunter [3] published a seminal paper “A Theory for Record Linkage” based on the assumption that the more similar two references

are, i.e. the closer they match, the more likely they are to be equivalent. The Fellegi-Sunter model is a mathematical model providing a theoretical framework for a computer oriented solution to the problem of recognizing references in two files that represent the same entity. The theory is based on the similarity (agreement) or dissimilarity (disagreement) between the values of certain attributes of the reference, e.g. name, address, etc.

It is difficult to overstate the significance that in the Fellegi-Sunter model is an implementation of pairwise logic where each record is compared with all other records. The number of comparisons required increases quickly as the number of records to be compared increases $O(n^2)$ – a quadratic computational complexity. The resources required to achieve the comparison, using Fellegi-Sunter alone, exceeds the capability of single threaded based entity resolutions even in today’s supercomputer environments.

A. Boolean Matching Rules

Match rules such as “score” or “weight” return an indication of the degree of similarity between a pair of values. Boolean rules, however, return an indicator that assumes only one of two states. The two states represent the conclusion: “matched” or “not matched”, 0 or 1, true or false. [4]. When referencing Boolean rules it is helpful to agree upon terminology. The basic unit of Boolean rules is often referred to as a “term”. In this case, term is the predicate serving to represent the comparison between a pair of attribute values. Terms may be aggregated via “AND” logic operators. Terms may invoke comparison functions extending well beyond a simple “exactly equal” comparison. Examples include Levenshtein edit distances, Q-Gram Tetrahedral Ratio, and Soundex. In a series of terms connected by “AND” logic, every term must return true in order for the rule to return true. Many implementations of Boolean matching rules also provide 2 or more sets of terms can also be connect via “OR” logic operators, with the provision that at least one of the terms is capable of returning a true indication when presented with values sufficiently similar to warrant a matched conclusion [5].

II. PROBLEM STATEMENT

Three main ER challenges occur when scaling to volumes of hundreds or thousands of millions of entity references:

Firstly, big volume data brings the storage and process performance challenge in every data processing including ER. Fortunately, technology has stepped up to the challenge with

cheaper and larger storage systems, distributed and parallel computing platforms, and cloud computing. However, these new technologies require changes in ways of thinking about data processing and the adoption of new tools and methodologies.

Secondly, the challenge of ER on big volume data is brought by the nature of ER. In ER, each reference must potentially be compared with every other reference, one pair at a time, in order to determine if the references are equivalent, i.e. represent the same entity. ER is subject to the $O(n^2)$ problem. $O(n^2)$ denotes the effort and resources needed to complete an algorithm or data process growing in proportion to the square of the number of records being processed. For example, the effort required to perform ER on 100 records is 4 times more than the effort of the same process on 50 records.

Last but not least, the transitive closure problem of performing ER on big volume data appears to be the most significant of the challenges. Transitive closure happens when a glue record link brings some references not directly equivalent together [6]. For instance, reference A matches reference B, implying A is equivalent to B. At the same time, reference B matches reference C, implying B is equivalent to C. Even though reference A may not match reference C, A and C are determined as equivalent because each are equivalent to the intermediate reference B. The general solution to transitive closure is to view the references with components as forming a graph in which the references are the vertices and the matching relationships form the edges. The maximally connected (strong) components of this graph formed by the transitive closure of the edges will partition the references into disjoint sets. Further, these components can be distributed and processed independently in a distributed processing environment. However, in the case of the data with big volume, a problem still exists. The standard algorithm for finding the maximally connected components relies upon recursion; and the graph size is limited by the memory available on a single processor.

III. METHODOLOGY

Hadoop MapReduce [7] is one of most popular software frameworks to process vast amounts of data in-parallel on large clusters of hardware in a reliable, fault tolerant manner. In the MapReduce architecture, a mapper function outputs key-value pairs. The shuffle brings together all of the key-value pairs into blocks that share the same key. These blocks are then processed by reducer functions. How to make this paradigm fit into ER scenario becomes a question.

The authors answer the question by perform **ER using hash keys**. In Boolean matching rules, comparison functions are used to determine the terms to be “TRUE” or “FALSE”. Two types of comparison functions are used. The first, Similarity functions, calculate similarity between attribute values, such as Levenshtein Edit Distance(LED), **Longest Common Substring**, and so on. The other functions are **Hash functions**, also called the encoding functions, which encode attribute values before values are used in comparison processes, for example, Soundex, NYSIIS, and Double-Metaphone etc [8].

We call the encoding attribute values as hash keys. Envision a hash key based upon one or more attributes taken from the entity reference being examined according to the Boolean matching rules. Assuming the hash key is sufficiently granular, hash keys then become equivalent to match keys when establishing linkages between references. As using hash keys to match the references in Hadoop MapReduce framework, the traditional pairwise comparison in ER has been avoided. So, we call the methodology presented as below as large scale ER without pairwise matching.



Figure 1: Work Flow of the large scale ER without Pairwise Matching

The general work flow of the large scale ER without pairwise matching is shown in Figure 1. Three main sections include standardize, match key generation, and transitive closure. The standardization section focuses on standardizing the **unstructured and raw records from input sources** to a standard form that can be used by later processes. The match key generation process generates hash keys according to the Boolean matching rules from the structured records. In the transitive closure, instead of using the recursive breadth first search algorithm in the standard algorithm [9], the authors have developed an iterative algorithm for the distributed processing. Figure 2 presents the MapReduce design for the large scale ER without pairwise matching methodology. The following sections will explain this figure in detail.

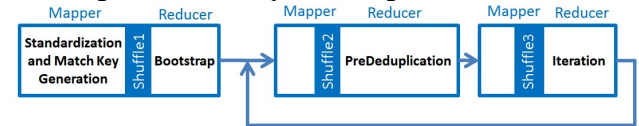


Figure 2: MapReduce Design of the large scale ER without ER Pairwise Matching

A. Standardization

Because the match keys will be generated on attribute values in the next step, it is necessary to transform the unstructured input data into a standard form so attribute values can be easily accessed in the later processes. In this system, the Compressed Document Set Architecture (CoDoSA) [10] is used as the standard form of references. CoDoSA uses short metadata tags in the string to separate and identify the subvalues encoded into an overall string. To save more disk space, the authors choose to use two digit of CoDoSA tag and not to use delimiters, which are located between metadata tags and data pieces. The relation of CoDoSA metadata tags and attributes are defined by users. Figure 3 shows an example of the XML which configures attributes and their CoDoSA metadata tags.

```

<Attributes>
  <Attribute Name="FirstName" Tag="AA"/>
  <Attribute Name="LastName" Tag="BB"/>
  <Attribute Name="DateOfBirth" Tag="CC"/>
  <Attribute Name="SSN" Tag="DD"/>
  <Attribute Name="@RefID" Tag="EE"/>
</Attributes>

```

Figure 3: An example XML to configure CoDoSA tags and attributes

This is not unsupervised like DWM

In the standardization process, there are two steps. Firstly, the unstructured input records split into pieces with their attributes, and then each piece finds its tag according to the attribute. Second, all pieces are then concatenated together to form a full string with CoDoSA format. Figure 4 presents an example of the standardization process with CoDoSA metadata configurations in Figure 3.

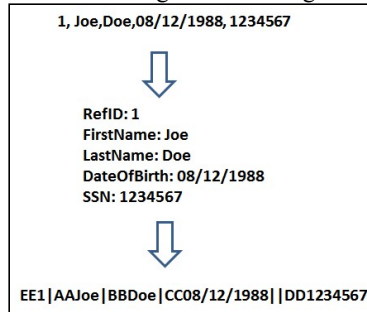


Figure 4: An example of the Standardization Process

B. Match Key Generation

Different than the normal Boolean rules, this methodology generates match keys of attribute values instead of executing similarity comparisons. Therefore, the structures to configure this kind of Boolean rules are slight different from the normal ones. Figure 5 shows an example configuration XML. The basic structures are the same with the normal Boolean rules. “Term”, the basic unit of Boolean rules in the traditional Boolean rules, has two attributes: “Item” and “Similarity”, but “Term” here has “Hash” attribute instead of the “Similarity” attribute.

```

<BooleanRules>
  <Rule Ident="1">
    <Term Item="FirstName" Hash="Soundex" />
    <Term Item="LastName" Hash="Exact" />
    <Term Item="SSN" Hash="Exact" />
  </Rule>
  <Rule Ident="2">
    <Term Item="FirstName" Hash="Exact" />
    <Term Item="LastName" Hash="Exact" />
    <Term Item="DateOfBirth" Hash="Scan(LR, Digit, 0, ToUpper, SameOrder)" />
  </Rule>
</BooleanRules>

```

Figure 5: A configuration XML for Generating Match Keys

The “Hash” attribute under a “Term” element defines a hash function, which will be used to generate a match key piece on the value of attribute defined in “Item”. The hash function could be phonetic encoding functions, such as Soundex [11], NYSIIS [12], Double-Metaphone [13], or string manipulation functions such as uppercasing, substring extraction, or SCAN hash algorithm [14].

The methodology also handles “AND” and “OR” logic differently from the traditional rules. For the terms sharing the “AND” logic, the hash pieces are concatenated together. For example, for the standard reference “EE1|AAJoe|BBDoe|CC08/12/1988||DD1234567”, the Rule 1 produces three pieces of hash from FirstName, LastName, and SSN. The three pieces are “J000”, “Doe”, and “1234567”. When combining pieces, CoDoSA metadata tags are added into each piece to avoid the future match key conflict. The match key generated by Rule 1 is “AAJ000BBDoeDD1234567”. Meanwhile, different rules containing different sets of “Term” elements generate different match keys for one reference to handle the “OR” logic. This means when one of another reference’s match keys matches the match key from either Rule 1 or Rule 2, a match happens. Therefore, for the example reference, the two match keys are: “AAJ000BBDoeDD1234567” and “AAJoeBBDoeCC08121988”.

When designing MapReduce steps in Hadoop, the authors put standardization and match key generation to the map side and bootstrap to the reduce side. Figure 6 presents an example of a mapper for standardization and match key generation. In this figure, the “R” in the figure is short for reference identifier. After the match keys have been generated, the attribute value pairs of the reference are not need for the actual transitive closure process and can be discarded for storage efficiency.

Each record generates one or more match keys except for the missing value cases. It is known that the reference which has missing attribute values should not match with any other references. Therefore, references with missing values should form individual single reference clusters in the final output. If these references are carried forward to participate in the transitive closure process with the empty match key, the references will form one large component that can significantly degrade the overall performance of the system, and more than that, the ER results are not correct because references with missing values are not meant to be equivalent.

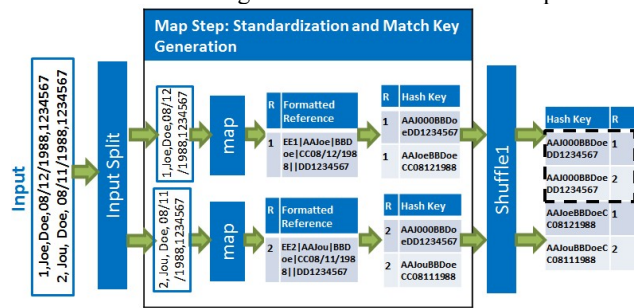


Figure 6: Example of Mapper for Standardization and Match Key Generation

C. Transitive Closure

When two or more rules are used to generate match keys, the transitive closure problem occurs. For example, if one more record “3, Jou, Doe, 08/11/1988, 1234568” is added into the system in Figure 6, the transitive closure will happen. This is because reference 2 and 3 are linked by the match key

“AAJouBBDDoeCC08111988” while the match key “AAJ000BBDDoeDD1234567” connects reference 1 and reference 2. A transitive closure algorithm is needed to establish that reference 1, 2, and 3 are linked in the final result set.

Standard transitive closure algorithms typically used to locate connected references (transitive closure of components expressed using graph terminology) rely upon recursion. To better fit the Hadoop MapReduce paradigm the authors design a new iterative algorithm for finding the link components of a match key graph. This new algorithm achieves transitive closure through the iterative, sequential processing of a large set of key-value pairs without the need to create large, in-memory data structures. Three parts in the algorithm include: an initial bootstrap phase, a pre-deduplication phase, and an iteration phase. As the work flow in Figure 7 demonstrates, the bootstrap step just processes once while the iteration phase and deduplication phases repeat until no more transitive closure change happens.



Figure 7: Workflow of Iterative Transitive Closure Algorithm

1) Bootstrap

The bootstrap phase is located in a reducer whose input is the result from shuffle 1. An example of a bootstrap reducer is presented in Figure 8. In the result from shuffle 1, the key is a match key and the value is the reference identifier (RefID). In the shuffle 1, the key-value pairs are sorted by match key to help to find the connected pairs of references in the bootstrap. The bootstrap reducer processes each group of RefIDs, per match key, in parallel. The group of RefIDs is the proto-component of the transitive closure graph. The match key-values are discarded, and only RefIDs are carried forward into the pre-deduplication and iteration phases. After getting the RefID group, the bootstrap reducer takes each proto-component and expands it into a set of key-value pairs where the key is a single RefID taken from the proto-component and the value is the entire proto-component. If a proto-component just has one RefID, the reducer will not generate any result for it, because there is no linkage. The Figure 9 shows the pseudo code of the bootstrap reducer.

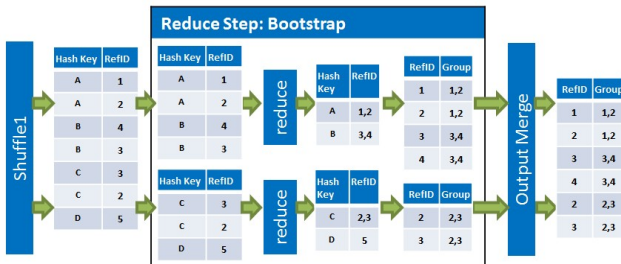


Figure 8: Example of a Reducer for Bootstrap

```

Bootstrap
reduce (key, <iterable> values)
  group ← ∅
  for each (value ∈ values)
    group.add(value)
  size ← group.length
  if(size > 1)
    for each (value ∈ group)
      emit(value, group)
  
```

Figure 9: Pseudo Code of the Bootstrap Reducer

2) Pre-DeDuplication

Due to the nature of the algorithm and the noise inherent in the match key graph, the same component may appear in different positions in the list. The deduplication processes are needed. The authors designed a pre-deduplication phase to support the deduplication. Actually, the real deduplication does not happen in this phase, and the deduplication actually occurs in the shuffle 3 after the pre-deduplication step. The pre-deduplication phase is designed to prepare for deduplication in the next phase.

In the MapReduce design of Figure 2, the outputs from the bootstrap phase flow into an identity mapper in Hadoop. In the identity mapper, every input key-value pair obtained from the input is written out. Because the identity mapper does not do anything, the mapper before the pre-deduplication in Figure 2 is left empty. After the identity mapper, the shuffle 2 sorts the output from the identity mapper in order by RefID, and feeds the data to a pre-deduplication reducer. In the pre-deduplication reducer as shown in Figure 10, the proto-components sharing the same RefID go into the same reducer, and merge (set union) to produce a new proto-component. The reducer then creates key-value pairs where the key is the first RefID in the new proto-component, and the value is the entire component. Because RefIDs are maintained in the proto-component sets in sorted order, if two components agree on the first RefID in the later shuffle 3, then both are the same component. The pseudo Code of the pre-deduplication Reducer is presented in Figure 11.

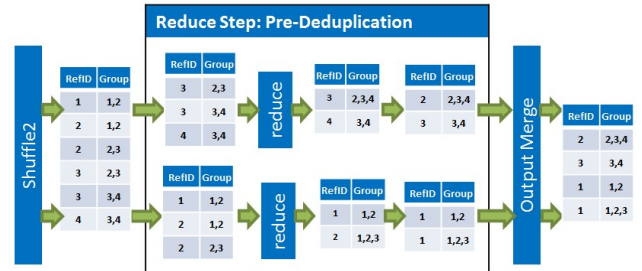


Figure 10: Example of a Reducer for Pre-DeDuplication

```

Pre-Deduplication
reduce (key, <iterable> valuelists)
  group ← ∅
  for each (valuelist ∈ valuelists)
    thisize ← valuelist.length
    if(thisize != previousize)
      boolean ischange ← True
      for each (value ∈ valuelist)
        group.add(value)
      presize ← thisize
    if(ismatch != True)
      Counter.NotChange.increment(1)
    else
      Counter.changes.increment(1)
  emit(group.first, group)

```

Figure 11: Pseudo Code of the Pre-DeDuplication Reducer

3) Iteration

Similarly with the pre-deduplication phase, the outputs from last phase go through an identity mapper first and then go into the shuffle 3 where the deduplication occurs. After the deduplication in shuffle 3, the Iteration reducer receives sets of proto-components as shown in Figure 12. Firstly, the proto-components sharing the same RefID go into the same reducer. These merge (set union) to produce a new proto-component. If the new proto-component is different from any of the input proto-components, it signals a “Change”. If none of the proto-components merge to form new proto-components, this condition is called a “NotChange”. If a new proto-component has a “Change” status, the iteration reducer will expands it into a set of key-value pairs where the key is a single RefID taken from the new proto-component and the value is the entire proto-component. Otherwise, the output of a new proto-component with “NotChange” status is just one key-value pair with first RefID as key and the entire component as value. The reason for the “NotChange” setting is that the cross-product key-value pairs of a new proto-component with “NotChange” no longer contribute to the transitive closure of other components. When some big components do not change at all in the whole process, but are still expanded, then occupy memory and disk space. If there are one or more “Change” components in this entire MapReduce iteration, this means transitive closure is not complete. The iteration step outputs the result to the pre-deduplication step to begin a new transitive closure process loop. The loops stop when all proto-components indicate a “NotChange” condition in the iteration step. The pseudo code of the iteration reducer is listed in Figure 13.

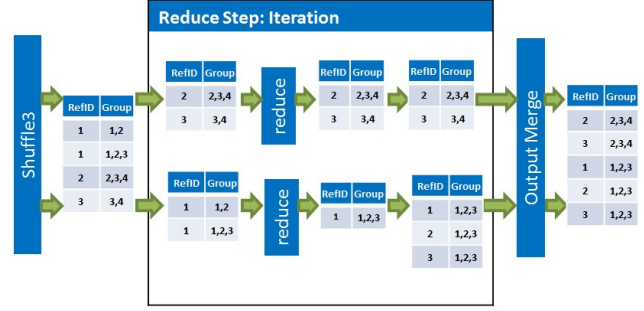


Figure 12: Example of a Reducer for Iteration

```

Iteration
reduce (key, <iterable> valuelists)
  group ← ∅
  for each (valuelist ∈ valuelists)
    thisize ← valuelist.length
    if(thisize != previousize)
      boolean ischange ← True
      for each (value ∈ valuelist)
        group.add(value)
      presize ← thisize
    if(ismatch != True)
      Counter.NotChange.increment(1)
      emit(group.first, group)
    else
      Counter.changes.increment(1)
      for each (value ∈ group)

```

Figure 13: Pseudo Code of the Iteration Reducer

4) Symetric Transitive Closure

An interesting, and somewhat surprising, observation was made while testing transitive closure processes. In some cases it may be appropriate to use the reference identifier, rather than the match key, as the closure variable. It is not difficult to reverse key generation logic such that the key-value pairs input to the closure bootstrap designate the reference identifier as the key with the match key as the value. Closure then aggregates match keys by reference rather than references by match key. Provided relatively few match keys are generated, closure on the reference IDs can be more efficient than closure by match key, as the cross products are smaller. Closure by reference identifier rather than match key does, however, come at a small price. After closure completes, it is necessary to recover the original reference value via two joins: the first join links the match key with the reference identifier and the second join links the reference identifier to the reference value.

IV. EXPERIMENTS

The authors tested the methodology for large scale ER without pairwise matching in Apache Hadoop 2.5.2. The data for testing was synthetically generated by SOG, a synthetic occupancy generator to support ER [15]. This data set is comprised of 84,515 records. This is not a large data set; however, the group sizes produced in the test performed are sufficient to stress both reference identifier closure and match key closure algorithms. The generation of match keys includes

configurations producing groups much larger than those used in practice for a data set of this size.

A. Comparison Between Pairwise and Non-Pairwise ER

To validate the accuracy of the non-pairwise ER methodology in this paper, the authors compare the results of five configurations using this methodology with using a pairwise ER matching engine, OYSTER [16]. In the ER results, the references marked with the same identifier are gathered into one cluster. The comparison result are listed in Table 1, and the number of total clusters are compared. Also, the Talburt-Wang index (TWi), a measurement to measure different ER results [17], is used to compare the results. In Table 1, the five configurations' TWi are all 1.0, which means the ER result from this methodology is the same with the ER result from OYSTER.

TABEL 1: COMPARISON BETWEEN PAIRWISE AND NON-PAIRWISE ER

Rule	Total Records	Total Clusters (Pairwise)	Total Clusters (Non-Pairwise)	TWi
1	84,515	20,067	20,067	1.0
2	84,515	28,847	28,847	1.0
3	84,515	82,813	82,813	1.0
4	84,515	84,092	84,092	1.0
5	84,515	84,466	84,466	1.0

B. Comparison Between Reference ID Closure and Match Key Closure

To find the difference between reference identifier closure and match key closure, the authors use two different configurations to demonstrate the advantage of the symmetric transitive closure on certain configurations. The performance comparison is listed in Table 2. In the table, "AVG of Group" means the average references' number of hash key groups. "SD of Group" represents the standard deviation for the references number of hash key groups. "Total RefID" is the total number of reference identifiers, while "Total Hash Keys" is the total number of hash keys. "Max Size after TC" is the number of references in the hash key group, which has the maximum references after transitive closure processes. "RefID Closure Time" and "Match Key Closure Time" represents the time used for running the reference identifier closure and match key closure respectively. The final column "Preference" is the preference transitive closure type for the rules.

TABEL 2: PERFORMANCE COMPARISON

Rule	AVG of Group	SD of Group	Total RefID	Total Hash Keys
1	3.4557	1.6571	84,515	48,914
2	7.0429	15.8580	84,515	24,000

Rule	Max Size after TC	RefID Closure Time	Match Key Closure Time	Preference
1	9	0:02:21	0:04:13	RefID
2	7,548	Failure	0:09:23	Index

Two configurations for match key generation in Table 2 are dramatically different and represent two extremes. Rule 1, represents a configuration producing relatively small components. The largest group after the TC process produces a maximum group of 9 references. This configuration marginally favors using reference IDs for closure. The difference in run time is primarily caused by the requirement for the extra join required to get back to reference IDs from the match keys. Configurations using many index rules will intensify this advantage as it reduces the compression offered by the match key approach.

In contrast, Rule 2 produces a maximum group of 7,548 references. This is the most extreme of the two configurations. It is a configuration that would be difficult, if not impossible, to perform in pairwise ER. Rule 2 causes the reference ID approach to run out of memory during processing. Reference ID TC was not capable of completing the TC even with a reducer memory configuration of 4 GB. The match key reduced the number of nodes in the graph, reduced memory consumption and successfully completed the TC process. Taking advantage of the symmetric nature of this transitive closure by using match keys is not only an optimization but can allow for the processing of data when the reference ID method will not work.

V. CONCLUSION AND OUTLOOK

A new methodology for large scale ER is brought up in this paper by using hash keys. Standardization, match key generation, and transitive closure are the three main part of this methodology. To overcome the limitation of traditional transitive closure, a new iterative transitive closure in MapReduce scenario is designed. To examine the methodology, the ER result from this new methodology is compared with the in-memory ER engine OYSTER. The result proves this new methodology without pairwise matching generated the same result with the pairwise ER method. One more interesting experiment is done to discover which method is optimal between reference identifier closure and match key closure. The authors found that different datasets and different rule configurations have different preferences when performing transitive closure.

In this paper, the authors tested the methodology on a synthetic dataset. Even though it can prove the methodology works, testing with real datasets is needed. In the future, the authors plan to test the methodology on a large, real dataset. Furthermore, more research work needs to be done on the two methods of transitive closure. Also, the author may explore platforms other than MapReduce to examine the methodology.

REFERENCES

- [1] M. G. de Carvalho, M. Goncalves, A. Laender and A. da Silva, "Learning to deduplicate," Chapel Hill, NC, 2006.
- [2] J. R. Talburt, Entity Resolution and Information Quality, San Francisco: Morgan Kaufmann, 2010.
- [3] I. P. Fellegi and A. B. Sunter, "A Theory for Record Linkage," Journal of the American Statistical Association, vol. 64, pp. 1183-1210, 1969.

- [4] S. E. Whang and H. Garcia-Molina, "Entity Resolution with Evolving Rules," in VLDB Endowment, 2010.
- [5] Y. Zhou, J. Talburt, F. Kobayashi and E. D. Nelson, "Implementing Boolean Matching Rules in an Entity Resolution System using XML Scripts," Las Vegas, 2011.
- [6] J. Zhang, R. Bheemavaram and W. N. Li, "Transitive Closure of Data Records: Application and Computation," 2006.
- [7] Apache, "Apache Hadoop," 2014. [Online]. Available: <http://hadoop.apache.org/>. [Accessed 15 July 2015].
- [8] P. Christen, Data Matching: Concepts and techniques for Record Linkage, Entity Resolution, and Duplication Detection, Springer, 2012.
- [9] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Pearson Education, 2011.
- [10] J. Talburt and E. Nelson, "CoDoSA: A Lightweight, XML-Based Framework for Integrating Unstructured Textual Information," 2009.
- [11] D. Holmes and M. McCabe, "Improving precision and recall for Soundex retrieval," 2002.
- [12] C. Borgman and S. Siegfried, "Getty's synonameTM and its cousins: A survey of applications of personal name-matching algorithms," Journal of the American Society for Information Science, vol. 43, pp. 459-476, 1992.
- [13] L. Philips, "The double-metaphone search algorithm.," C/C++ User's Journal, vol. 18(6), 2000.
- [14] J. Talburt, "Oyster v3.3 Reference Guide 2013 03 18.pdf," 25 July 2013. [Online]. Available: http://downloads.sourceforge.net/project/oyster/OYSTER_3.3/Documentation/Oyster%20v3.3%20Reference%20Guide%202013%2003%2018.pdf. [Accessed 16 July 2015].
- [15] R. J. Talburt, Y. Zhou and S. Y. Shivaiah, "SOG: A synthetic occupancy generator to support entity resolution instruction and research," in International Conference on Information Quality, Potsdam, 2009.
- [16] Y. Zhou and J. R. Talburt, "OYSTER: An Open Source Entity Resolution System Supporting Identity Information Management," in ID360 - The Global Forum on Identity, Austin, 2012.
- [17] J. Talburt, R. Wang, K. Hess and E. Kuo, "An Algebraic Approach to Data Quality Metrics for Entity Resolution Over Large Datasets," in Information Quality Management: Theory and Applications, IGI Global, 2007, pp. 1-22.